

数据结构练习题 1

1. 编一 C 程序, 它根据读入的数据构造有向图 G, 并输出 G 的邻接矩阵及 G 的拓扑序列。图的输入形式为 $i_0 \ j_0 \ i_1 \ j_1 \ i_2 \ j_2 \ \dots i_m \ j_m \ -1 \ -1$ (-1, -1 为有向边输入结束标记, 其余的值都 ≥ 0 且 $< n$ 。) n, 它们都是整数, n 是有向图 G 的顶点个数且 $40 > n > 0$, 顶点名字依次是 $V_0, V_1, V_2, \dots, V_{n-1}$; $i_k \ j_k$ 表示有向图 G 的有向边 $\langle V_{i_k}, V_{j_k} \rangle$ 。

(注: 程序的可执行文件名必须是 e1.exe, 存于你的账号或其 debug 目录下。)

```
#include "stdafx.h" //对.MFC (微软的基础类库) 文件必须用 (因为其中以 c++形式封装
//了 Windows 的 API), 对.CPP (用 c++语言编写的源代码文件) 文件就不须要用
#include <stdio.h> //基本输入输出库
#include <stdlib.h> //库文件
1. void exit(int); //exit(0)非正常结束程序运行, exit(1) 正常结束程序运行
#define MaxVerNum 40 //最大顶点数设为 40
typedef struct { //结构类型
    int arcs[MaxVerNum][MaxVerNum]; //定义二维数组类型的邻接矩阵
    int vexnum; //定义顶点数
} MGraph; //实现的是邻接矩阵存储图
2. void Input(MGraph *g) //建立邻接矩阵
{
    int n, i, j; //初始化顶点数 n
    printf("请输入顶点个数, 顶点个数必须在[1...%d]\n", MaxVerNum); //输出提示信息
    scanf("%d", &n); //键入十进制数字 n
    if(n < 0 || n > MaxVerNum) {
        printf("顶点的个数必须在[1...%d]\n", MaxVerNum); //输出提示信息
        exit(-1); // (非零) 正常结束程序运行
    }
    g->vexnum = n; //用邻接矩阵初始化有向边
    for(i = 0; i < n; i++)
        for(j = 0; j < n; j++) //书 P144( $v_i, v_j$ )或 $\langle v_i, v_j \rangle$ 不是 E(G)的边则初始化邻接
            g->arcs[i][j] = 0; //矩阵元素为 0 如果是网则=0 改为=32767 (无穷)
    printf("请输入边的顶点 i, j 必须在[0...%d]\n", n-1); //输出提示信息
    do {
        scanf("%d%d", &i, &j); //输入一条边的两个顶点编号 i, j
        if(i == -1 && j == -1) //如果 i, j 为有向边结束标记
            break; //跳出循环
        if(i < 0 || i >= n || j < 0 || j >= n) { //如果输入有向边超出范围
            printf("边的顶点 i, j 必须在[0...%d]\n", n-1); //输出提示信息
            continue; //继续循环
        }
        g->arcs[i][j] = 1; //书 P144( $v_i, v_j$ )或 $\langle v_i, v_j \rangle$ 是 E(G)的边, 则邻接矩阵元素为 1
    } while(1);
} //以上构建有向图
3. void juzhen(MGraph *g) //构建邻接矩阵
{
```

```

int c=0,i,j; //c=count 对顶点计数
printf("\n 图 G 的邻接矩阵如下:\n");
for(i=0;i<g->vexnum;i++)
    for(j=0;j<g->vexnum;j++)
        {printf("%d  ",g->arcs[i][j]);
          if(++c%g->vexnum==0)// (+1 后的顶点数) 被顶点除后==0 即矩阵每个顶点都有了
              printf("\n");
        }
}

4.void cal_in(MGraph *g,int in[]) //计算图中顶点的入度
{
    int i,j;
    for(i=0;i<g->vexnum;i++) //初始化数组, 每个元素的入度 in[i]值为 0——P164 上
        in[i]=0;
    for(i=0;i<g->vexnum;i++) //扫描每个顶点 vi 的出边表, 统计每个顶点的入度数
        for(j=0;j<g->vexnum;j++)
            if(g->arcs[i][j])
                in[j]++; //统计入度数
}

5.void TopSort(MGraph *g,int in[]) //用链栈对图进行拓扑排序输出拓扑序列 P164 下
{ //建立入度为 0 的顶点栈——用指针数组模拟栈
    int s=-1,c=0,i,j; // c=count 对输出顶点计数, s 为栈顶指针
    printf("\n 图 G 的拓扑排序如下:\n");
    for(i=0;i<g->vexnum;i++)
        if(in[i]==0){ //若入度为 0
            in[i]=s; //使 in[i]入栈
            s=i; //i 为图 g 中的第一个邻接点
        } //依次将入度为 0 的顶点压入堆栈
    while(s!=-1){ //若 s!=-1 表示栈非空
        printf("V%d ",s); //输出一个顶点
        c++; //输出顶点数+1
        i=s; //出栈
        s=in[s];
        for(j=0;j<g->vexnum;j++){ //扫描顶点出边表
            if(g->arcs[i][j])
                in[j]--; //vj 的入度减 1, 相当于删除边<vi, vj>
            if(in[j]==0){ //减 1 后, 若 vj 的入度为 0, 则新的 vj 入栈
                in[j]=s; //入栈
                s=j; //j 为图 g 中顶点 i 的下一个邻接点
            }
        }
    }
    if(c<g->vexnum){ //当输出的顶点数小于图中的顶点数时, 说明图有回路, 排序失败
        printf("\n 图 G 中包含环路!\n"); //输出提示信息
    }
}

```

```

    }
}
6.void main()           //主函数名，无返回值
{
    1.MGraph g;         //用邻接矩阵存储图
        int InDegree[MaxVerNum]; //按照无先驱顶点优先思想，degree[]存放结点的入度
    2.Input(&g);         //输入邻接矩阵各元素（顶点）
    3.juzhen(&g);        //构建邻接矩阵
    4.cal_in(&g,InDegree); //计算图中顶点的入度
    5.TopSort(&g,InDegree); //顶点排序
}
2. 编一 C 程序，它能读入两组整数（每组整数都以-9999 为结束标记，个数都不大于 1000），
并以从小到大的次序输出既在第一组整数中也在第二组整数中的所有整数（同一个整数不能
输出两次）。（输入时，两个相邻的整数用空格隔开）。
（注：程序的可执行文件名必须是 e2.exe，存于你的账号或其 debug 目录下。）
#include "stdafx.h"      //对.MFC 文件必须有，对.CPP 文件就不须要用
#include<iostream.h>     //用于支持 C++I/O 操作的头文件
#define MAX 1000         //最大数设为 1000
int countA;
int countB;
int countC;
1.void output(int a[],int n)
{
    //建立一个输出用的整型数组
    int i;                //数组中间变量为 i
    for(i=0;i<n;i++){
        cout<<a[i]<<" "; //向输出数组中插入字符：“ ”；
    }
    cout<<endl;          //换行（另一种是\n）
}
2.void read(int a[],int *n) //读数组元素
{
    int i,x;              //定义两个中间变量 i, x
    *n=0;                 //n 的值从“0”开始
    cout<<"请输入一串整数:(以-9999 结束,个数都不大于 1000)"<<endl; //输出“提示”换行
    for(i=0;i<MAX;i++){
        cin>>x;           //从键盘获取输入数据给缓冲存储器 x
        if(x==-9999)      //如果 x 值为结束标记-9999
            break;        //满足以上条件时停止循环
        a[i]=x;           //不满足以上条件时执行：将 x 的值赋给 a[i]
        ++(*n);           //*n 的值++1
    }
}
3.void sort(int a[],int n) //排序 a[]
{

```

```

int i,j,t,ok=0;
for(i=1;i<n;i++){
    ok=1;
    for(j=0;j<n-i;j++){
        if(a[j]>a[j+1]){ //若 a[j]>a[j+1], 则两元素互换位置
            t=a[j];
            a[j]=a[j+1];
            a[j+1]=t;
            ok=0;
        }
    }
    if(ok)
        break;
}
cout<<"排序后的数组为:"<<endl; //输出提示信息换行
output(a,n); //输出 a[]
cout<<"完成排序!"<<endl; //输出提示信息换行
}

4.void jiaoji(int a[],int b[],int c[]) //用 c[]求 a[]、b[]的交集
{
    int i,j,k=0; // i,j,k 为各矩阵中的指针
    for(i=0;i<countA;i++){ //对于每个 i 的具体元素值
        for(j=0;j<countB;j++){ //j 要从 0~B 比较一下有没有与 i 相同值的元素
            if(a[i]==b[j]){ //若 b[j]=a[i]
                c[k++]=a[i]; //则 a[i]元素值赋给 c[]中 k 元素的位置, 然后 k+1
                countC++; //c[]中的元素数+1
            }
        }
    }
    cout<<"两个数组中相同的元素有序数组为:"<<endl; //输出提示信息换行
    output(c,k); //输出交集 c[], c 中 k 个元素
}

5.void clearsame(int a[]) //清除 a[]中具有相同值的元素
{
    int i,j,k;
    for(i=0;i<countC;i++){ //从第一个元素 i=0 开始
        for(j=i+1;j<countC;j++){ //对于每个 i 值从第二个元素 j=i+1 开始一直比较到 j=c
            if(a[i]==a[j]){ //如果有相等的即 a[i]==a[j]
                for(k=j;k<countC;k++){ //k=j 开始到 k<c
                    a[k]=a[k+1]; //即 c[]中有 a[k]=a[k+1]
                }
                k--; //则 k-1——即以下删除 c 中的一个元素
                j--; //j-1
                countC--; // countC-1
            }
        }
    }
}

```

```

    }
}
cout<<"删除相同元素中重复的元素后的数组为:"<<endl; //输出提示信息换行
output(a,countC);
}
6.void main()                                //主函数名，无返回值
{
    int a[MAX],b[MAX],c[MAX];                //定义三个矩阵
    int t=1;                                  //设初值 t=1
    while(t){                                  //当 t=1 时，在屏幕输出以下 9 行文字
        cout<<"*****欢迎使用此系统*****"<<endl;
        cout<<"1:输入数组数据"<<endl;
        cout<<"2:数组排序"<<endl;
        cout<<"3:两个数组中相同的元素"<<endl;
        cout<<"4:删除相同元素中重复的元素"<<endl;
        cout<<"5:输出所需的结果数据"<<endl;
        cout<<"0:退出系统程序"<<endl;
        cout<<"请选择你所需要的操作"<<endl;
        cout<<"*****谢谢使用此系统*****"<<endl;
        cin>>t; //从标准输入流对象（通常指键盘）读取的数即上述 0~5 操作之一传送给 t
        switch(t){
            case 1:                            //若从键盘读入的数字为 1 时，读入数组数据
                cout<<"请输入第一个数组:"<<endl;        //输出提示信息换行
                read(a,&countA);                        //读入 A[]
                cout<<"请输入第二个数组:"<<endl;        //输出提示信息换行
                read(b,&countB);                        //读入 B[]
                break;
            case 2:                            //若从键盘读入的数字为 2 时，数组排序
                cout<<"第一个数组排序:"<<endl;          //输出提示信息换行
                sort(a,countA);                        //A[]排序
                cout<<"第二个数组排序:"<<endl;          //输出提示信息换行
                sort(b,countB);                        //B[]排序
                break;                          //停止循环
            case 3:                            //若从键盘读入的数字为 3 时，两个数组中相同的元素
                cout<<"两个数组中相同的元素:"<<endl;    //输出提示信息换行
                jiaoji(a,b,c);                        //求相同元素值的交集
                break;                              //停止循环
            case 4:                            //若从键盘读入的数字为 4 时，删除相同元素中重复的元素
                cout<<"删除相同元素中重复的元素:"<<endl; //输出提示信息换行
                clearsame(c);                          //删 c[]中相同值的元素
                break;                              //停止循环
            case 5:                            //若从键盘读入的数字为 5 时，输出所需的结果数据在屏上显示
                cout<<"输出所需的结果数据:"<<endl; //输出所求的 c[]
                output(c,countC);

```

```

        break; //停止循环
    case 0: //若从键盘读入的数字为 0 时, 退出系统程序
        cout<<"退出系统程序"<<endl; //输出提示信息换行
        break; //停止循环
    default: //以上全不符合
        cout<<"选择错误!"<<endl; //输出提示信息换行
        break; //停止循环
    }
}
}

```

数据结构练习题 2

1. 编一 C 程序, 它能读入两组整数 (每组整数都是 66 个整数), 它们分别是下三角矩阵 A 和下三角矩阵 B 的按行优先排列的元素 (A 和 B 的其它元素均为零)。计算并输出矩阵 A 与 B 的乘积。——P93 正 6

(注: 程序的可执行文件名必须是 e1.exe, 存于你的账号或其 debug 目录下。)

```

#include "stdafx.h" //对.MFC 文件必须用, 对.CPP 文件就不须要用
#include <stdio.h> //基本输入输出库
#define MAX 66 //调试的值可以改小
#define M 11 //可以根据上面的值相应的改小, 公式: n*(n+1)/2=MAX.
1. void juzhen(int a[],int b[],int C[][M]) //构建三个矩阵, a、b 分别为一维数组
{ //存放矩阵 A、B 的下三角元素值, C 存放 A 和 B 的乘积
    int i,j,k;
    int A[M][M],B[M][M]; //建三个空矩阵 A、B、C
    for(i=0;i<M;i++){
        for(j=0;j<M;j++){
            A[i][j]=0;
            B[i][j]=0;
            C[i][j]=0;
        }
    }
    k=0;
    for(i=0;i<M;i++) //数组值赋给 A,B;
        for(j=0;j<=i;j++)
        {
            A[i][j]=a[k];
            B[i][j]=b[k];
            k++;
        }
    for(i=0;i<M;i++) //计算两矩阵 A、B 的乘积
        for(j=0;j<M;j++)
            for(k=0;k<M;k++)
                C[i][j]+=A[i][k]*B[k][j];
}

```

```

2.void main()                                //主函数名，无返回值
{
    int i,j,t=0;
    int a[MAX],b[MAX],c[M][M]; //定义三个矩阵
    printf("请输入第一组整数:\n"); //输出提示信息换行
    for(i=0;i<MAX;i++) //向矩阵中一行一行地输入数字
        scanf("%d",&a[i]);
    printf("请输入第二组整数:\n"); //输出提示信息换行
    for(j=0;j<MAX;j++) //每行中再一列一列地输入数字
        scanf("%d",&b[j]);
    juzhen(a,b,c); //求 a[] × b[]放入 c[]
    printf("输出的矩阵乘积为:\n"); //输出提示信息换行
    for(i=0;i<M;i++) //向矩阵中一行一行地输入 a[] × b[]数字
        for(j=0;j<M;j++){ //每行中再一列一列地输入 a[] × b[]数字
            printf("%-4d",c[i][j]); //左对齐 4 列十进制数
            if(++t)%M==0 //矩阵达到 M 行 M 列时 (+1 后的 t 被 M 除后为 0)
                printf("\n"); //换行
        }
    }
}

```

2. 编一 C 程序，它能对输入的一串整数（不多于 1000 个，以-9999 为结束标记）到数组 a 中，再对 a 的元素进行直接插入排序（从小到大排序），输出排序结果和所用关键字比较次数。（输入时，两个相邻的整数用空格隔开）。

（注：程序的可执行文件名必须是 e2.exe，存于你的账号或其 debug 目录下。）

```

#include "stdafx.h" //对.MFC 文件必须用，对.CPP 文件就不须要用
#include <stdio.h> //基本输入输出库
#define MAX 1000 //最大数设为 1000
void main() //主函数名，无返回值
{
    int i=1,j,n,t=0,a[MAX];
    printf("请输入一个整数串个数必须在[1...%d]，以-9999 作为结束标记\n",MAX); //提示
    do{
        scanf("%d",&a[i]); //在 a[i]中输入各十进制元素的数值
        if(a[i]==-9999) //如果 a[i]值为结束标记-9999
            break; //满足以上条件时停止循环
        i++; //不满足以上条件即 a[i]!= -9999 时，继续 i+1 循环
    }while(9);
    n=i-1; //书 P170
    for(i=2;i<=n;i++) //i 从 2 开始
        if(a[i]<a[i-1]){ //若 a[i]<a[i-1]若 a[i]>=有序区中所有的 key，则 a[i]不动
            a[0]=a[i]; //把 a[i]的值赋给 a[0]——当前记录复制为哨兵
            j=i-1; //将 i-1 赋给 j
            do{ //找插入位置
                a[j+1]=a[j]; //将 a[j]的值赋给 a[j+1]——记录后移
                j--; //j-1
            }while(j>0);
        }
    }
}

```

```

        t++; //t+1, t 为关键字的比较次数
    }while(a[0]<a[j]); // 当 a[0]<a[j]
    a[j+1]=a[0]; // 将 a[0]值赋给 a[j+1] ——将 a[i]插入到正确位置
}
printf("整数串排序的结果为:\n"); //输出提示信息换行
for(i=1;i<=n;i++) //满足条件时
    printf("%-4d",a[i]); //将 a[i] 左对齐用 4 列十进制数输出
printf("\n"); //不满足以上条件时——换行
printf("排序中所用的关键字比较次数为:%d\n",t); //提示-用十进制表示的关键字次数 t
}

```

数据结构练习题 3

1. 编一 C 程序，它能根据输入的二叉树前序和中序序列来构造该二叉树，并能输出该二叉树的后序序列和该二叉树叶的结点的个数以及该二叉树高度。（输入次序是：表示前序序列的字符串、表示中序序列的字符串）。

（注：程序的可执行文件名必须是 e1.exe，存于你的账号或其 debug 目录下。）

```

#include"stdafx.h" //对.MFC 文件必须用，对.CPP 文件就不须要用
#include<stdio.h> //基本输入输出库
#include <stdlib.h> //库文件
#include<string.h> //关于字符数组（串）函数定义的头文件
1.void exit(int); //exit(0)非正常结束程序运行，exit(1) 正常结束程序运行
#define MAX 1000 //最大数设为 1000
typedef struct node{ //定义链表结点（元素）
    char d; //字符型的数字域
    struct node *lchild,*rchild; //结构型结点（元素），含左子树、右子树
}Tnode; //构建具有结点的二叉树
2.void MK(char in[],char is,char ie,char pre[],char pres,char pree,Tnode **r)
//in, is, ie 中序，始，长，pre, pres, pree 前序，始，长，Tnode **r 指针序列的指针 r
{ //MK: 一个库函数，做段基址+偏移地址运算的宏——设置一个远指针——求实际地址
    int i; //中间用的整型变量 i
    if(is>ie||pres>pree) //若中序始地址>中序长度或前序始址>前序长
        *r=NULL; //则指针序列*r 为空
    else{
        *r=(Tnode *)malloc(sizeof(Tnode)); //申请*r 的存储空间
        (*r)->d=pre[pres]; //将前序序列的始元素放入*r 所指的数字域 d 中
        for(i=is;i<=ie;i++){ //i 初值为 is（第一个元素）开始，在 i≤ie 中循环，每循环 i+1
            if(in[i]==pre[pres]){ //若符合 pres 元素=i 所指中序 in 的结点,即在 in 中找根结点
                MK(in,is,i-1,pre,pres+1,pres+i-is,&(*r)->lchild); //中序序列，中序始址，i-1，前
                //序序列，前序始址+1，前序始址+i-is，*r 所指的地址为左子树
                MK(in,i+1,ie,pre,pres+i-is+1,pree,&(*r)->rchild); //中序序列，i+1，中序长度，前
                //序序列，前序始址+i-is+1，前序长度，*r 所指的地址为右子树
                break; //停止循环
            }
        }
        if(i>ie){ //若 i 所指结点地址>中序长度

```

```

        printf("输入错误!\n");//输出提示信息换行
        exit(1);           // (非零) 正常结束程序运行
    }
}
}
}
3.void postorder(Tnode *r)//书 P116 倒 10, 后序遍历二叉链表算法
{
    if(r){
        postorder(r->lchild);//后序遍历左子树
        postorder(r->rchild);//后序遍历右子树
        printf("%c",r->d); //访问根结点
    }
}
int leaf(Tnode *r)           //叶结点个数
{
    if(r==NULL)              //如果 r 为空, 返回 0——没有叶子
        return 0;
    else
        if(r->lchild==NULL&& r->rchild==NULL)
            return 1; // 如果 r 所指的左、右子树均为空, 则返回 1 即 1 个叶子——即根
        else //左、右子树均非空时, 返回左子树叶子数+右子树叶子数
            return leaf(r->lchild)+leaf(r->rchild);
}
int height(Tnode *r)         //高度
{
    int h1,h2;               //定义两个高度数 h1, h2
    if(r==NULL)
        return 0;           //若 r 为空, 则高度为 0
    else{
        //高度不为 0 时 (r 不空)
        h1=height(r->lchild); //左子树高度赋给 h1
        h2=height(r->rchild); //右子树高度赋给 h2
        return 1+(h1>h2?h1:h2);//返回 1 (即根) +左右子树中高度值大者
    }
}
4.void main()                //主函数名, 无返回值
{
    Tnode *r=NULL;           //建立一个空的二叉树链表, 指针为 r
    char in[MAX],pre[MAX];    //定义字符型的中序、前序序列的最大长度
    printf("请输入前序序列:\n");//输出提示信息换行
    gets(pre);                //用读键盘函数获取 (键入) 前序序列
    printf("请输入中序序列:\n");//输出提示信息换行
    gets(in);                  //用读键盘函数获取 (键入) 中序序列
    MK(in,0,strlen(in)-1,pre,0,strlen(pre)-1,&r);
}

```

```

//中序, 起始为 0, 中序串长-1, 前序, 起始为 0, 前序串长-1, 指针 r 所指的地址
//MK: 一个库函数, 做段基址+偏移地址运算的宏—设置一个远指针—求实际地址
printf("\n 后序遍历序列为:\n"); //输出提示信息换行
postorder(r); //输出后序序列
printf("\n 叶结点的个数为:%d\n",leaf(r)); //输出提示信息换行
printf("\n 高度为:%d\n",height(r)); //输出提示信息换行
}

```

2. 编一 C 程序, 它能读入一串 (n 个) 整数 (以-9999 为结束标记), 并判断第 1 个整数在后(n-1)个整数中出现的次数, 再输出该次数。(输入时, 两个相邻的整数用空格隔开)。

(注: 程序的可执行文件名必须是 e2.exe, 存于你的账号或其 debug 目录下。)

```

#include"stdafx.h" //对.MFC 文件必须用, 对.CPP 文件就不须要用
#include<stdio.h> //基本输入输出库
#define MAX 100 //最大数设为 100
void main() //主函数名, 无返回值
{
    int a[MAX]; //c 用来记录第 1 个整数在后 n-1 个整数中出现的次数
    int i=0,c=0,t,k; //t 邻时存放第 1 个整数, k 为数组 a[MAX]的长度
    printf("请输入一串整数, 以-9999 作为结束标记\n");//输出提示信息换行
    do{
        scanf("%d",&a[i]); //在 a[i]的地址中, 输入各元素的十进制数值
        if(a[i]==-9999) //当 a[i] 元素的值为-9999 (结束标记) 时
            break; //满足以上条件时停止循环
        i++; //a[i]≠-9999 即不满足以上条件时继续 i+1 循环
    }while(1);
    k=i;
    printf("\n 这个整数串的第一个数是:%d\n",a[0]); //输出提示信息换行
    t=a[0]; //将第一个整数值赋给 t
    for(i=1;i<k;i++){ //在 i=1~k-1 范围内, 即从第二个开始
        if(a[i]==t) //查找与第一个整数相同的数
            c++; //每找到一个 c+1
    }
    printf("\n 第一个整数在后(n-1)个整数中出现的次数为:%d\n",c); //输出提示信息换行
}

```

数据结构练习题 4

1. 编一 C 程序, 它能根据输入的二叉树中序和后序序列来构造该二叉树, 并能输出该二叉树的前序序列和该二叉树的度为 2 的结点的个数并能判断该二叉树是否为二叉排序树 (若是输出 Yes; 否则输出 No)。(输入次序是: 表示中序序列的字母串、表示后序序列的字母串)。

(注: 程序的可执行文件名必须是 e1.exe, 存于你的账号或其 debug 目录下。)

```

#include"stdafx.h" //对.MFC 文件必须用, 对.CPP 文件就不须要用
#include<stdio.h> //基本输入输出库
#include<string.h> //关于字符数组 (串) 函数定义的头文件
#include<stdlib.h> //库文件
1. void exit(int); //exit(0)非正常结束程序运行, exit(1) 正常结束程序运行

```

```

#define MAX 1000           //最大数设为 1000——建一个空的二叉树链表
typedef struct node{       //结构型链表结点
    char d;                //字符型数据域
    struct node *lchild,*rchild; //定义构造型的结点结构, 含左、右子树
}Tnode;

2. void MK(char in[],char is,char ie,char post[],char posts,char poste,Tnode **r)
//in, is, ie 中序, 始, 长, post, posts, poste 后序, 始, 长, Tnode **r 指针序列的指针 r
{ //MK: 一个库函数, 做段基址+偏移地址运算的宏——设置一个远指针——求实际地址
    int i; // 中间用的整型变量 i
    if(is>ie||posts>poste) //若中序始地址>中序长度
        *r=NULL; //则指针序列*r 树为空
    else{
        *r=(Tnode *)malloc(sizeof(Tnode)); //申请*r 的存储空间
        (*r)->d=post[poste]; //将后序序列的长度放入*r 所指的数字域 d 中
        for(i=is;i<=ie;i++)//i 初值为 is (第一个元素) 开始, 在 i<=ie 中循环, 每循环 i+1
            if(in[i]==post[poste]){ // 在中序 in 的结点, 中找根
                MK(in,is,i-1,post,posts,posts+i-is-1,&(*r)->lchild); //中序序列, 中序始址, i-1,
                //后序序列, 后序始址+i-is-1, *r 所指的地址为左子树——确定左子树
                MK(in,i+1,ie,post,posts+i-is,poste-1,&(*r)->rchild); //中序序列, i+1, 中序长度,
                //后序序列, 后序始址+i-is, 后序长度-1, *r 所指的地址为右子树——确定右子树
                break; //不满足条件停止循环
            }
        if(i>ie){ //若 i 所指结点地址>中序长度
            printf("输入错误!\n"); //输出提示信息换行
            exit(1); // (非零) 正常结束程序运行
        }
    }
}

2. void preorder(Tnode *r) //书 P115 倒 12, 采用二叉链表存储结构, 并设结点值为字符型
{
    if(r){ //建立前序序列
        printf("%c",r->d); //访问根结点
        preorder(r->lchild); //前序遍历左子树
        preorder(r->rchild); //前序遍历右子树
    }
}

int one(Tnode *r) //求度为一的结点个数
{
    if((r==NULL)||(!r->lchild)&&(!r->rchild))
        //若 r 为空, 或逻辑非 r (若 r≠0, 则逻辑非 r=0) 指针指向左子树同时指向右子树
        return 0; //返回空——树为空——没有叶子结点
    if((r->lchild)&&(r->rchild))// 若 r 指针同时指向左子树、右子树——表示有叶子结点
        return one(r->lchild)+one(r->rchild);
    /
    //返回左子树度为一的数+右子树度为一的数——即计算度为一的结点个数
}

```

```

else //否则
    return 1+one(r->lchild)+one(r->rchild);
//返回 1+左子树度为一的数+右子树度为一的数——即计算子树度为一的树的结点总个数
}
int two(Tnode *r) //求度为二的结点个数
{
    if((r==NULL)||(!r->lchild)&&(!r->rchild))
        //若 r 为空, 或逻辑非 r (若 r≠0, 则逻辑非 r=0) 指针指向左子树同时指向右子树
        return 0; // 返回空——树为空——没有叶子结点
    if((r->lchild)&&(r->rchild)) // 若 r 指针同时指向左子树、右子树——表示有叶子结点
        return 1+two(r->lchild)+two(r->rchild); //
    //返回 1+左子树度为二的数+右子树度为二的数——即计算子树度为二的树的结点总个数
    else{ //否则
        return two(r->lchild)+two(r->rchild);
        //返回度为二的左子树数+度为二的右子树数——即计算度为二的结点个数
        /*if((r->lchild)&&(!r->rchild)) //若 r 指针指向左子树, 同时逻辑非 r 指针指向右子树
            return two(r->lchild); //返回 r 所指左子树度为二的数
        else //否则
            return two(r->rchild);*/ //返回 r 所指右子树度为二的数
    }
}
int check(Tnode *r) //检查*r 所指的二叉树
{
    if((r==NULL)||(!r->lchild)&&(!r->rchild)) // 若 r 为空或非 r 所指的左子树和右子树
        return 1; //返回 1——只有根结点
    if((r->lchild)&&(!r->rchild)){ //若 r 指向左子树或同时非 r 指向右子树
        if(r->lchild->d<r->d) //若 r 所指的左子树所指的数据域 d<r 所指的数据域 d
            return check(r->lchild); //返回所检查的 r 所指的左子树
        else //否则
            return 0; //返回空——树为空——没有叶子结点
    }
    else{ //否则
        if((!r->lchild)&&(r->rchild)){ //若非 r 指针指向左子树同时 r 指针指向右子树
            if(r->rchild->d>r->d) //若 r 指向右子树所指的数据域 d>r 所指的数据域 d
                return check(r->rchild); //返回所检查的 r 所指的右子树
            else //否则
                return 0; //返回空——树为空——没有叶子结点
        }
        if((r->lchild->d<r->d)&&(r->rchild->d>r->d))
            //若 r 所指的左子树所指的数据域 d<r->d, 同时 r 所指右子树指向的 d>r->d
            return (check(r->lchild)&&check(r->rchild));
            //返回所检查的 r 所指的左子树和检查 r 所指的右子树
        else //否则
            return 0; //返回空——树为空——没有叶子结点
    }
}

```

```

    }
}
void main() //主函数名，无返回值
{
    1.Tnode *r=NULL; //建立一个*r 所指的空树
    2.char in[MAX],post[MAX];
    printf("请输入中序序列:\n"); //输出提示信息换行
    gets(in); //用读键盘函数获取（键入）中序序列
    printf("请输入后序序列:\n"); //输出提示信息换行
    gets(post); //用读键盘函数获取（键入）后序序列
    MK(in,0,strlen(in)-1,post,0,strlen(post)-1,&r);
    3.printf("\n 该二叉树的前序遍历序列为:\n"); //输出提示信息换行
    preorder(r); //列出前序序列
    printf("\n 该二叉树的一度结点个数为:%d\n",one(r)); //输出提示信息换行
    printf("\n 该二叉树的二度结点个数为:%d\n",two(r)); //输出提示信息换行
    printf("\n 该二叉树是否为二叉排序树(若是输出 Yes,否则输出 No):\n"); //输出提示信息
    if(check(r))
        printf("Yes\n");
    else
        printf("No\n");
}

```

2. 编一 C 程序，它能读入一串整数（以-9999 为结束标记），再以与输入次序相反的次序输出这串整数（输入、出时，两个相邻的整数用空格隔开）。

（注：程序的可执行文件名必须是 e2.exe，存于你的账号或其 debug 目录下。）

```

#include "stdafx.h" //对.MFC 文件必须用，对.CPP 文件就不须要用
#include<stdio.h> //基本输入输出库
#define MAX 100 //最大数设为 100
void main() //主函数名，无返回值
{
    int i=0,k,a[MAX];
    printf("请输入一串整数，以-9999 作为结束标记:\n");//输出提示信息换行
    do{
        scanf("%d",&a[i]); //在 a[i]的地址中，输入各元素的十进制数值
        if(a[i]==-9999) //当 a[i] 元素的值为-9999（结束标记）时
            break; //满足以上条件时停止循环
        i++; // a[i]≠-9999 即不满足以上条件时继续 i+1 循环
    }while(1);
    k=i;
    printf("这串整数的相反次序是:\n");//输出提示信息换行//
    for(i=k-1;i>=0;i--) //从 i=k-1 到 i=0 反向输出这串整数，每输出一个，i-1 循环
        printf("%d",a[i]); //全部输出反向 a[i]
}

```

数据结构练习题 5

1. 编一 C 程序，它根据输入数据构造带权无向图 G，并输出 G 的最小生成树。图的输入形式为 n V0 Vi0 w0 V1 Vi1 w1 V2 Vi2 w2 ...Vi Vin wn -1 -1 -1 (-1, -1,-1 为输入结束标记，其余的值都 ≥ 0 且 $\leq n$)，它们都是整数，且 $100 > n > 0$ 。

(注：程序的可执行文件名必须是 e1.exe，存于你的账号或其 debug 目录下。)

```
#include"stdafx.h"           //对.MFC 文件必须用，对.CPP 文件就不须要用
#include<stdio.h>             //基本输入输出库
#include<stdlib.h>            //库文件
1.void exit(int);             //exit(0)非正常结束程序运行，exit(1) 正常结束程序运行
#define MAX 100               //最大数设为 100
#define MAXINT 32767          //最大数设为无穷大
2.void read(int a[][MAX],int n) //95 正 3
{
    int i,j,w;
    do{
        scanf("%d%d%d",&i,&j,&w);//读入顶点数和权值
        if(i==-1||j==-1||w==-1) //若有向边权值=输入结束标记
            break;              //跳出循环
        if(i<0||i>=n||j<0||j>=n||w<=0||i==j){//数入有向边、权值超出范围
            printf("\n 重新输入:权值 w 必须正确， i 和 j 必须在[0..%d]\n",n-1); //输出提示
            continue;           //继续循环
        }
        a[i][j]=w;              //置矩阵元素权值
    }while(1);
}
3.void MST(int a[][MAX],int n) //书 p95 正 3 三元组
{
    int c=0,i,j,min_i,min_j,min_v,sum=0; // min_i,min_j,min_v 为三元组
    printf("生成的最小生成树为:\n");    //输出提示信息换行
    a[0][0]=-1;                          //表示数组为空，建一个每元素为三元组的空数组
    for(c=0;c<n-1;c++){                  //求 T 的 n-1 条树边
        min_v=MAXINT;                    //值为最大
        min_i=-1;                        //行下标为空
        min_j=-1;                        //列下标为空
        for(i=0;i<n;i++){
            for(j=0;j<n;j++){
                if(i!=j&&(a[i][i]==-1&&a[j][j]!=-1||a[i][i]!=-1&&a[j][j]==-1)&&a[i][j]<min_v){
                    min_v=a[i][j];      //最小路径的值
                    min_i=i;             //最小路径行下标
                    min_j=j;             //最小路径列下标
                }
            }
        }
    }
}
```

```

        if(min_i!=-1){
            printf("<V%-2d,V%-2d>",min_i,min_j);//%-md: 按十进制 m 位左对齐处理整数
            sum+=min_v;
            a[min_i][min_i]=a[min_j][min_j]=-1;
        }
        else{
            printf("\n 图 G 是不连通的!\n");    //输出提示信息换行
            exit(1);                            //正常结束运行
        }
    }
    printf("\n 各边的权值之和为:%d\n",sum);    //输出提示信息换行
}
4.void main()                                //主函数名, 无返回值
{
    int a[MAX][MAX];
    int i,j,vnum;
    printf("请输入顶点个数和边及边的权值:\n"); //输出提示信息换行
    scanf("%d",&vnum);
    if(vnum<1||vnum>MAX){
        printf("错误:顶点个数必须在:[1...%d]\n",MAX); //输出提示信息换行
        exit(1);                                        //正常结束程序运行
    }
    for(i=0;i<vnum;i++)
        for(j=0;j<vnum;j++)
            a[i][j]=MAXINT;
    read(a,vnum);
    MST(a,vnum);
}

```

2. 编一 C 程序, 它能根据读入的数据构造有向图 G, 并输出 G 的 DFS (深度优先搜索算法) 遍历序列 (从 V0 开始), 图的输入形式为 n V0 Vi0 V1 Vi1 V2 Vi2...Vi Vin -1 -1 (-1, -1 为输入结束标记, 其余的值都 ≥ 0 且 $\leq n$), 它们都是整数, 且 $100 > n > 0$ 。

(注: 程序的可执行文件名必须是 e2.exe, 存于你的账号或其 debug 目录下。)

```

#include "stdafx.h"    //对.MFC 文件必须用, 对.CPP 文件就不须要用
#include<stdio.h>      //基本输入输出库
#include<stdlib.h>     //库文件
1.void exit(int);      //exit(0)非正常结束程序运行, exit(1) 正常结束程序运行
#define MAX 100        //最大数设为 100
typedef struct{
    int a[MAX][MAX];    //边表
    int vexnum;         //顶点个数
}MGraph;               //建立图的邻接矩阵
typedef enum{False,True} Boolean;
Boolean visited[MAX];
2.void Input(MGraph *g)

```

```

{
    int i,j,n; //初始化顶点
    printf("请输入顶点个数, 必须在[1...%d]范围内:\n",MAX); //输出提示信息换行
    scanf("%d",&n); //键入十进制顶点数 n
    if(n<0||n>MAX){
        printf("顶点个数必须在[1...%d]范围内!\n",MAX); //输出提示信息换行
        exit(1); //正常结束程序运行
    }
    g->vexnum=n; //初始化有向边
    for(i=0;i<n;i++)
        for(j=0;j<n;j++)
            g->a[i][j]=0; //P144( $v_i, v_j$ )或 $< v_i, v_j >$ 不是  $E(G)$ 的边, 初始化邻接矩阵元素值为 0
    printf("请输入边的结点 i,j,必须在[0...%d]:\n",n-1); //输出提示信息换行
    do{
        scanf("%d%d",&i,&j); //输入一条边的两个顶点编号 i, j
        if(i== -1&&j== -1) //如果为有向边, 输入结束标记
            break; //跳出循环
        if(i<0||i>=n||j<0||j>=n){ //输入有向边超出范围
            printf("边的结点 i,j,必须在[0...%d]范围内,请重新输入:\n",n-1); //输出提示信息
            continue; //继续循环
        }
        g->a[i][j]=1; //P144( $v_i, v_j$ )或 $< v_i, v_j >$ 是  $E(G)$ 的边, 初始化邻接矩阵元素值为 1
    }while(1);
}

3.void DFSM(MGraph *g,int i)
{
    //从顶点  $v_i$  出发深度优先搜索遍历图  $G$  (邻接矩阵结构) ——P149 上
    int j;
    printf("V%-3d",i); //假定访问顶点  $v_i$  以输出该顶点的序号代之
    visited[i]=True; //标记  $v_i$  已访问过
    for(j=0;j<g->vexnum;j++) //依次搜索  $v_i$  的每个邻接点
        if(g->a[i][j]==1&&!visited[j])
            DFSM(g,j); //若  $(v_i, v_j) \in E(G)$ , 且  $v_i$  未被访问过, 则从开始递归调用
    }

4.void main() //主函数名, 无返回值
{
    MGraph g; //用邻接矩阵存储图
    Input(&g); //输入邻接矩阵各元素 (顶点)
    printf("\n 图 G 的 DFS 遍历序列如下所示:\n"); //输出提示信息换行
    DFSM(&g,0); //输出深度优先搜索遍历图  $G$ 
    printf("\n");
}

```